

透過 AP2 和 UCP 保護代理商務

來源：<https://codelabs.developers.google.com/next26/adk-agent-commerce?hl=zh-tw>

步驟數：10

建構 ADK 代理，使用 UCP 進行商家通訊，並使用 AP2 確保付款安全，以便預訂電影票。

目錄

1. [總覽](#)
2. [瞭解 UCP 和 AP2 通訊協定](#)
3. [設定環境](#)
4. [定義代理](#)
5. [建構代理程式工具：探索和瀏覽](#)
6. [建構代理程式工具：結帳和付款](#)
7. [讓內容可執行](#)
8. [使用 ADK Web 執行代理](#)
9. [清除所用資源](#)
10. [恭喜！🎉](#)

1. 總覽

在本程式碼研究室中，您將執行 **ADK** 代理，使用兩種開放原始碼商務通訊協定，向多個電影院商家預訂電影票：

- **UCP (通用商務通訊協定)**：代理程式探索商家、搜尋目錄及管理結帳流程的標準。
- **AP2 (Agent Payments Protocol)**：這項通訊協定會使用以加密編譯方式簽署的授權，確保付款授權安全無虞且可驗證。

CineAgent 試用版應用程式會連線至兩個具有不同功能的模擬電影院商家 (選位、特殊格式和付款方式)，並從搜尋到付款，安排完整的預訂流程。

課程內容

- 如何透過 `/.well-known/ucp` 設定檔探索 UCP 商家
- ADK 代理如何使用 UCP 搜尋目錄及建立結帳程序
- AP2 授權書 (CartMandate、PaymentMandate) 如何確保交易安全
- 端對端 UCP 和 AP2 通訊協定如何確保代理式電子商務安全

軟硬體需求

- 已啟用計費功能的 Google Cloud 雲端專案
- 網路瀏覽器，例如 [Chrome](#)
- Python 3.11 以上版本

本程式碼研究室適合對 Python 和 Google Cloud 有基本認識的中階開發人員。完成這個程式碼研究室大約需要 15 分鐘。

本程式碼研究室建立的資源費用應低於 \$5 美元。

2. 瞭解 UCP 和 AP2 通訊協定

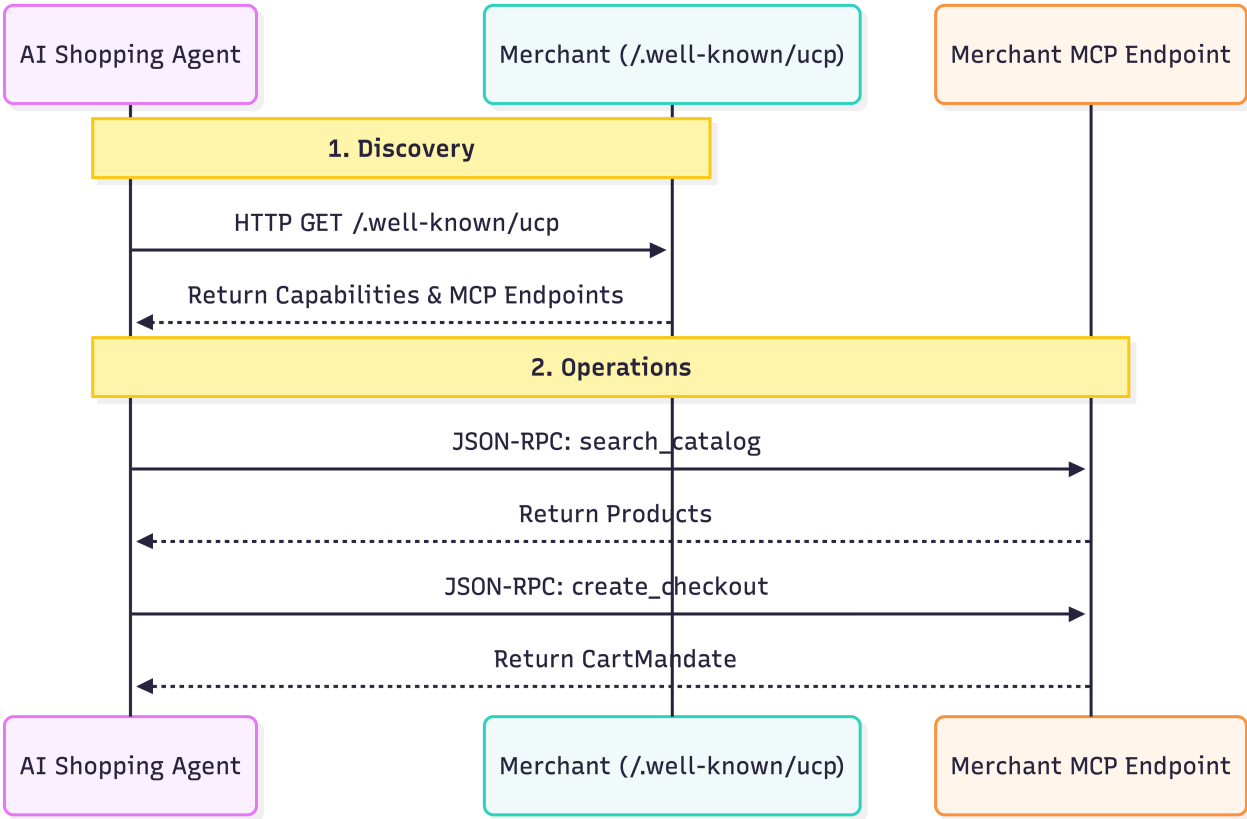
在深入瞭解如何建構代理程式前，我們先來瞭解這兩個通訊協定，這兩個通訊協定可確保代理功能商務安全無虞。

通用商務通訊協定 (UCP)

UCP 會統一 AI 代理與商家互動的方式。這項功能引進標準化資源模型，解決代理程式必須為每間商店學習自訂 API 的問題。

運作方式：

1. 探索：每個符合 UCP 規範的商家都會在標準位置 (`/.well-known/ucp`) 公開設定檔。範例：[Everlane 的 UCP 端點](#)。代理程式讀取這個設定檔時，會尋找：
 - **功能**：商家支援的獨立核心功能，例如目錄搜尋或結帳。
 - **服務**：用於交換資料的低層級通訊層。範例：REST API、MCP (Model Context Protocol)、A2A (Agent2Agent Protocol)。
 - **擴充功能**：如果商家需要特殊行為，可以在這個設定檔中定義自訂擴充功能。
2. 作業：探索完成後，代理程式會使用提供的服務端點執行作業。在本程式碼研究室中，我們使用 **Model Context Protocol (MCP)** 做為服務傳輸方式。代理程式會對這個端點發出 JSON-RPC 2.0 呼叫，以叫用探索到的功能：搜尋產品、建立結帳程序及完成購買。



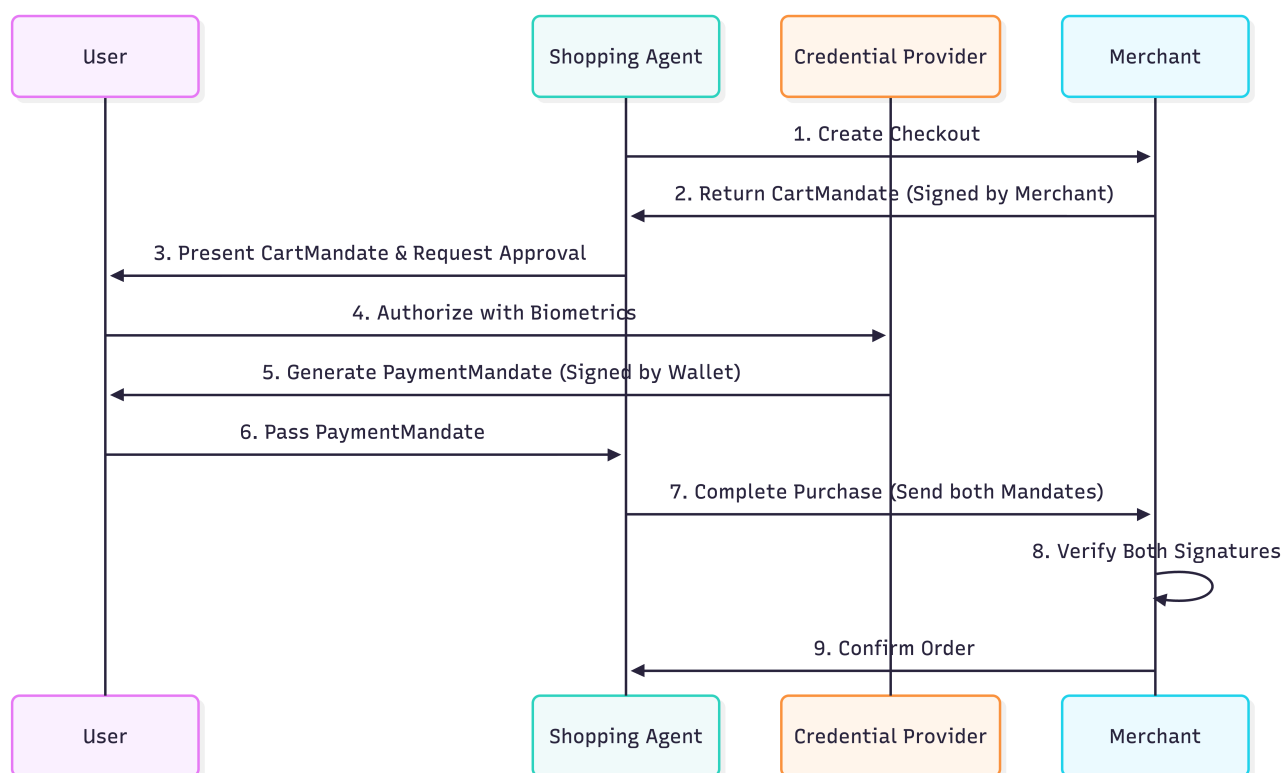
Agent Payments Protocol (AP2)

AP2 標準可讓代理人代表使用者授權付款。解決客服人員處理敏感付款憑證時的安全問題。

運作方式：

1. **購物車授權**：當代理程式使用 UCP 通訊協定建立結帳程序時，商家會傳回 `CartMandate`。這是包含購物車詳細資料和商家加密簽章的 JSON 物件。這項服務可確保你不會買貴。發出授權後，商家就無法變更價格。
2. **付款委託書**：驗證購物車內容後，使用者 (或代表使用者的代理程式) 會建立 `PaymentMandate` 來授權付款。這個 `PaymentMandate` 會參照 `CartMandate`，並包含使用者的加密簽章 (或授權權杖)。
3. **雙重簽名驗證**：商家會收到兩份授權書。他們會驗證 `CartMandate` 上的簽名是否為自己的簽名，以及 `PaymentMandate` 上的簽名是否為使用者的簽名。如果兩者都有效，交易就會繼續進行。

這套「雙重鎖定」系統可確保商家不會超額收費，代理商也不會未經授權就花費。在正式環境中，這些授權會使用 **SD-JWT** (選擇性揭露 JWT) 來保護使用者隱私權。



步驟 3 · 約 5 分鐘

3. 設定環境

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

設定 Google Cloud 專案

建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，[選取或建立 Google Cloud 專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

啟動 Cloud Shell

Cloud Shell 是在 Google Cloud 中運作的指令列環境，已預先載入必要工具。

1. 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。
2. 連至 Cloud Shell 後，請驗證您的驗證：

```
gcloud auth list
```

3. 確認專案已設定完成：

```
gcloud config get project
```

4. 如果專案未如預期設定，請設定專案：

```
export PROJECT_ID=<YOUR_PROJECT_ID>
gcloud config set project $PROJECT_ID
```

存取 Gemini 模型

在 Cloud Shell 殼層環境中，複製並貼上下列指令。這項設定會啟用 Cine Agent 使用的 Gemini 模型存取權。

```
export GOOGLE_CLOUD_PROJECT=$PROJECT_ID
export GOOGLE_CLOUD_LOCATION=global
export GOOGLE_GENAI_USE_VERTEXAI=True
```

設定目錄結構

複製並貼上下列指令，為代理程式建立新目錄：

```
mkdir -p agent_payments
cd agent_payments
```

安裝依附元件

Google Cloud Shell 已預先安裝 `uv`，可管理環境和依附元件。

1. 在 `agent_payments` 資料夾的根目錄建立 `pyproject.toml` 檔案，並加入以下內容。這個檔案定義了專案中繼資料和依附元件。

```
[project]
name = "agent-payments-demo"
version = "0.1.0"
description = "CineAgent booking agent using UCP and AP2"
requires-python = ">=3.11"
dependencies = [
    "google-adk>=1.29.0",
    "google-genai>=1.27.0",
    "fastapi>=0.115.0",
    "uvicorn>=0.34.0",
    "httpx>=0.28.0",
    "ap2 @ git+https://github.com/google-agentic-commerce/AP2.git@main",
    "ucp-sdk @ git+https://github.com/Universal-Commerce-Protocol/python-sdk.git@main",
]
```

2. 執行下列指令，建立虛擬環境並安裝所有依附元件：

```
uv sync
```

3. 啟動 `uv` 建立的虛擬環境：

```
source .venv/bin/activate
```

步驟 4 · 約 3 分鐘

4. 定義代理

撰寫任何工具邏輯之前，請先在名為 `agent.py` 的檔案中定義代理程式本身。這個代理程式會做為電影訂票流程的自動化調度管理工具。

建立 `agent.py`：

```
"""CineAgent — movie ticket booking agent using UCP and AP2."""

from google.adk.agents import Agent
from google.adk.tools import FunctionTool
from agent_payments.tools import (
    discover_theaters,
    search_movies,
    get_movie_detail,
    create_checkout,
    complete_purchase,
)

root_agent = Agent(
    model="gemini-3.1-pro-preview",
    name="cineagent",
    description="Movie ticket booking agent using UCP and AP2.",
    instruction="""You are CineAgent, a movie ticket booking assistant.

You help users find and book movie tickets across multiple theaters
using UCP (Universal Commerce Protocol) and AP2 (Agent Payments).

**Your tools:**
- discover_theaters: Find theaters and what they support
- search_movies: Search movies across all theaters
- get_movie_detail: Get showtimes at a specific theater
- create_checkout: Start a checkout session
- complete_purchase: Finalize with AP2 mandate signing

**Rules:**
- Always call discover_theaters first if you haven't yet
- Keep responses concise — summarize and suggest next steps
- Prices from tools are in cents (1500 = $15.00)
- Never invent data — only state what tools return
""",
    tools=[
        discover_theaters,
        search_movies,
        get_movie_detail,
        create_checkout,
        FunctionTool(complete_purchase, require_confirmation=True),
    ],
)
```


解釋程式碼

讓我們來細分這個代理定義的內容：

- `model="gemini-3.1-pro-preview"`：我們使用最新的 Gemini Pro 搶先版模型，進行複雜的推論和工具使用。
- `instruction`：這是引導代理程式行為的提示。這項提示會明確告知代理程式使用 UCP 和 AP2、列出可用工具，並設定重要規則，例如「絕不編造資料」和「價格以分計算」。
- `tools`：這是 Python 函式清單 (我們會在下一個步驟中建構)，代理程式可根據使用者的要求選擇呼叫這些函式。
- `require_confirmation`：您可以使用 `FunctionTool(my_function,require_confirmation=True)` 包裝任何工具。觸發後，代理程式會暫停並等待簡單的「是」或「否」核准，然後再執行工具。在這裡，代理會在執行 `complete_purchase` 工具前暫停，等待真人確認。

工具清單

代理程式定義會宣告我們需要建構的內容。每項工具都會對應至 UCP 或 AP2 通訊協定中的特定作業：

工具	用途	通訊協定動作
<code>discover_theaters</code>	尋找商家及其服務	查詢 <code>/.well-known/ucp</code>
<code>search_movies</code>	搜尋各商家的目錄	JSON-RPC 至 MCP 端點
<code>get_movie_detail</code>	取得特定商家的電影場次	JSON-RPC 至 MCP 端點
<code>create_checkout</code>	啟動結帳工作階段	JSON-RPC 至 MCP 端點
<code>complete_purchase</code>	授權付款並完成訂單	簽署 AP2 授權書並傳送給 MCP

Gemini 模型會根據對話內容，決定何時呼叫各項工具。接下來，我們的工作是在 `tools.py` 中實作每個工具的功能。

步驟 5 · 約 5 分鐘

5. 建構代理程式工具：探索和瀏覽

現在，我們來實作代理程式用來瀏覽及探索電影的工具。每個工具都會包裝 UCP 作業。

建立名為 `tools.py` 的新檔案，然後複製並貼上下列程式碼：

```
"""Agent tools — each one wraps a UCP or AP2 operation."""

import asyncio
import json

from .ucp import UCPCClient
from .ap2 import AP2Handler

# Initialize clients directly
_merchant_urls = ["http://localhost:8081", "http://localhost:8082"]
_ucp = UCPCClient()
_ap2 = AP2Handler()
```

瞭解設定

為讓本程式碼實驗室專注於建構代理程式，我們使用了兩個輔助類別 `UCPCClient` 和 `AP2Handler`，稍後會介紹這兩個類別。

- 簡介：這些是我們為本程式碼研究室建立的手寫輔助類別，用於模擬與模擬商家互動。由於官方 **UCP** 和 **AP2 SDK** 尚未推出，我們使用這些輔助程式來彌補差距。在正式環境中，您可以使用正式版 SDK。
- 目前請將這些物件視為輔助物件：
 - `_ucp.discover(url)`：擷取商家檔案。
 - `_ucp.mcp_call(url, method, params)`：將 JSON-RPC 2.0 要求傳送至商家 MCP 端點。

探索劇院

這項工具是 UCP 流程的第一步。並找出現有的商家和支援的項目。

新增至「`tools.py`」：

```
async def discover_theaters() -> str:
    """Discover available theater merchants and their capabilities via UCP."""
    theaters = []
    for url in _merchant_urls:
        info = await _ucp.discover(url)
        theaters.append(
            {
                "url": url,
                "name": info["name"],
                "capabilities": info["capabilities"],
                "payment_handlers": info["payment_handlers"],
            }
        )
    return json.dumps(theaters, indent=2)
```

此步驟的目標：

1. 這項函式會疊代伺服器提供的商家網址清單。在本程式碼研究室的後續章節中，我們會設定兩個模擬商家。
2. 針對每個網址，系統會呼叫 `_ucp.discover(url)`，這會觸及 `/.well-known/ucp` 端點。
3. 並將名稱、功能和付款處理常式收集到摘要清單中。
4. 並以 JSON 字串的形式傳回清單，供代理程式讀取。

搜尋電影

這項工具會搜尋所有已發現的商家，並合併結果。這點非常重要，因為同一部電影可能會在多個戲院以不同格式 (IMAX、Dolby) 播放，且價格不同。

新增至「`tools.py`」：

```
async def search_movies(query: str = "") -> str:
    """Search for movies across all theaters. Use '' to browse all."""
    all_movies = {}
    for url, merchant in _ucp.merchants.items():
        result = await _ucp.mcp_call(url, "search_catalog", {"query": query})
        for product in result.get("products", []):
            mid = product["id"]
            if mid not in all_movies:
                all_movies[mid] = {
                    "id": mid,
                    "title": product["title"],
                    "categories": product.get("categories", []),
                    "theaters": {},
                }
            showtimes = []
            for v in product.get("variants", []):
                opts = {
                    o["name"]: o["value"]
                    for o in v.get("selected_options", [])
                }
                showtimes.append(
                    {
                        "id": v["id"],
                        "format": opts.get("format", "Standard"),
                        "time": opts.get("time", ""),
                        "price": v.get("price", {}),
                        "seats": v.get("availability", {}).get(
                            "seats_available", 0
                        ),
                    }
                )
            all_movies[mid]["theaters"][url] = {
                "name": merchant["name"],
                "showtimes": showtimes,
            }
    return json.dumps(list(all_movies.values()), indent=2)
```

此步驟的目標：

1. 這個函式會逐一檢查所有找到的劇院。
2. 將搜尋目錄 JSON-RPC 要求 (`_ucp.mcp_call(url, "search_catalog", {"query": query})`) 傳送至商家 MCP 端點。
3. 接著，系統會進行一些清理作業，剖析結果以找出電影及其「變體」(代表特定放映時間和格式)。依 ID 將電影分組，避免使用者看到重複的電影項目。

取得電影詳細資料

這項工具會取得特定電影在特定戲院的完整目錄詳細資料。

新增至「`tools.py`」：

```
async def get_movie_detail(movie_id: str, merchant_url: str) -> str:
    """Get detailed showtimes for a movie at a specific theater."""
    result = await _ucp.mcp_call(
        merchant_url, "lookup_catalog", {"product_id": movie_id}
    )
    return json.dumps(result, indent=2)
```

此步驟的目標：

- 這會呼叫商家 MCP 端點的 `lookup_catalog` 方法，並傳遞特定 `movie_id`。這會傳回詳細資訊，例如特定戲院的放映時間和座位供應情形。
-

步驟 6 · 約 5 分鐘

6. 建構代理程式工具：結帳和付款

這些工具會處理結帳和購買流程。此時 AP2 通訊協定就能派上用場，確保交易安全無虞。

建立結帳頁面

這項工具會在特定劇院的特定放映時間啟動結帳程序。

新增至「`tools.py`」：

```
async def create_checkout(
    merchant_url: str, showtime_id: str, quantity: int = 1
) -> str:
    """Create a checkout session for tickets at a theater."""
    result = await _ucp.mcp_call(merchant_url, "create_checkout", {
        "checkout": {
            "line_items": [
                {"item": {"id": showtime_id}, "quantity": quantity}
            ],
            "context": {"country": "US", "currency": "USD"},
        }
    })
    return json.dumps(result, indent=2)
```

此步驟的目標：

1. 並呼叫商家 MCP 端點的 `create_checkout` 方法。
2. 並傳遞使用者要求的 `showtime_id` 和 `quantity`。
3. 商家會傳回包含 **AP2 CartMandate** 的 JSON 物件。

注意：如果您檢查回應資料，`ap2.cart_mandate` 會包含 `merchant_authorization` 欄位。這是商家鎖定報價的加密簽章。設定後即無法變更！

完成購買程序

這項工具會執行三項操作：

1. 我們會從結帳程序取得 **CartMandate**，並進行驗證(模擬)。
2. 建立並簽署 **PaymentMandate** (模擬)。
3. 將簽署的委託書傳送給商家，完成購買程序。

新增至「`tools.py`」：

```

async def complete_purchase(
    checkout_id: str, merchant_url: str, payment_method: str = "card"
) -> str:
    """Complete purchase with AP2 payment authorization."""
    # 1. Get the CartMandate from the checkout
    checkout = await _ucp.mcp_call(
        merchant_url, "get_checkout", {"checkout": {"id": checkout_id}}
    )
    cart_mandate = _ap2.process_cart_mandate(checkout)
    if not cart_mandate:
        return {"error": "No cart mandate – checkout may have expired"}

    # 2-3. Create and sign the PaymentMandate
    # In production, this call would trigger a user prompt (biometric or device auth)
    # via the AP2 Wallet SDK. In this demo, it just computes a mock SHA-256 hash.
    payment_mandate = _ap2.create_payment_mandate(cart_mandate, payment_method)

    # 4. Send both mandates to complete the purchase
    result = await _ucp.mcp_call(merchant_url, "complete_checkout", {
        "checkout": {
            "id": checkout_id,
            "payment": {
                "instruments": [{
                    "handler_id": f"card_{merchant_url.split(':')[0][-1]}",
                    "type": "card",
                }],
            },
        },
        "ap2": {"payment_mandate": payment_mandate},
    })
    return json.dumps(result, indent=2)

```

為什麼需要兩份授權書？CartMandate 可確保商家在報價後無法變更價格。PaymentMandate 可確保代理程式不會在未經同意的情況下向使用者收費。流程如下：

Merchant locks price -> User authorizes charge -> Merchant verifies both -> Order completes。

檢查點：完整 `tools.py`

完整的 `tools.py` 應包含 5 個工具函式，以及 UCP 和 AP2 用戶端的模組層級初始化。確認是否像這樣：

```

"""Agent tools — each one wraps a UCP or AP2 operation."""

import asyncio
import json

from ucp import UCPClient
from ap2 import AP2Handler

# Initialize clients directly
_merchant_urls = ["http://localhost:8081", "http://localhost:8082"]
_ucp = UCPClient()
_ap2 = AP2Handler()

async def discover_theaters() -> str:
    """Discover available theater merchants and their capabilities via UCP."""
    theaters = []
    for url in _merchant_urls:
        info = await _ucp.discover(url)
        theaters.append(
            {
                "url": url,
                "name": info["name"],
                "capabilities": info["capabilities"],
                "payment_handlers": info["payment_handlers"],
            }
        )
    return json.dumps(theaters, indent=2)

async def search_movies(query: str = "") -> str:
    """Search for movies across all theaters. Use '' to browse all."""
    all_movies = {}
    for url, merchant in _ucp.merchants.items():
        result = await _ucp.mcp_call(url, "search_catalog", {"query": query})
        for product in result.get("products", []):
            mid = product["id"]
            if mid not in all_movies:
                all_movies[mid] = {
                    "id": mid,
                    "title": product["title"],
                    "categories": product.get("categories", []),
                    "theaters": {},
                }
            showtimes = []
            for v in product.get("variants", []):
                opts = {
                    o["name"]: o["value"]
                    for o in v.get("selected_options", [])
                }
                showtimes.append(
                    {

```

```

        "id": v["id"],
        "format": opts.get("format", "Standard"),
        "time": opts.get("time", ""),
        "price": v.get("price", {}),
        "seats": v.get("availability", {}).get(
            "seats_available", 0
        ),
    },
}

all_movies[mid]["theaters"][url] = {
    "name": merchant["name"],
    "showtimes": showtimes,
}

return json.dumps(list(all_movies.values()), indent=2)

async def get_movie_detail(movie_id: str, merchant_url: str) -> str:
    """Get detailed showtimes for a movie at a specific theater."""
    result = await _ucp.mcp_call(
        merchant_url, "lookup_catalog", {"product_id": movie_id}
    )
    return json.dumps(result, indent=2)

async def create_checkout(
    merchant_url: str, showtime_id: str, quantity: int = 1
) -> str:
    """Create a checkout session for tickets at a theater."""
    result = await _ucp.mcp_call(merchant_url, "create_checkout", {
        "checkout": {
            "line_items": [
                {"item": {"id": showtime_id}, "quantity": quantity}
            ],
            "context": {"country": "US", "currency": "USD"},
        }
    })
    return json.dumps(result, indent=2)

async def complete_purchase(
    checkout_id: str, merchant_url: str, payment_method: str = "card"
) -> str:
    """Complete purchase with AP2 payment authorization."""
    # 1. Get the CartMandate from the checkout
    checkout = await _ucp.mcp_call(
        merchant_url, "get_checkout", {"checkout": {"id": checkout_id}}
    )
    cart_mandate = _ap2.process_cart_mandate(checkout)
    if not cart_mandate:
        return {"error": "No cart mandate – checkout may have expired"}

    # 2-3. Create and sign the PaymentMandate

```



```
# In production, this call would trigger a user prompt (biometric or device auth)
# via the AP2 Wallet SDK. In this demo, it just computes a mock SHA-256 hash.
payment_mandate = _ap2.create_payment_mandate(cart_mandate, payment_method)

# 4. Send both mandates to complete the purchase
result = await _ucp.mcp_call(merchant_url, "complete_checkout", {
    "checkout": {
        "id": checkout_id,
        "payment": {
            "instruments": [{
                "handler_id": f"card_{merchant_url.split(':')[ -1]}",
                "type": "card",
            }],
        },
    },
    "ap2": {"payment_mandate": payment_mandate},
})
return json.dumps(result, indent=2)
```

步驟 7 · 約 2 分鐘

7. 讓內容可執行

如果是實際的 UCP 商家，您只要將網址提供給服務專員即可。在本程式碼研究室中，我們需要兩項項目才能在本機進行測試：

1. **模擬商家**：模擬 UCP 端點的本機伺服器，方便您進行測試
2. **通訊協定輔助程式** - UCP 和 AP2 的精簡 HTTP 包裝函式 (在正式版中，官方 SDK 會取代這些函式)

注意：您不需要仔細閱讀這段程式碼。這些檔案會模擬實際基礎架構和 SDK 提供的內容。直接複製。

通訊協定輔助程式

UCP 和 AP2 目前沒有用戶端 SDK，這兩個檔案會處理 HTTP 管道。

建立 `ucp.py`：

```

"""UCP client – discovers merchants and calls their MCP tools."""

import uuid
import httpx

class UCPClient:
    def __init__(self):
        self.client = httpx.AsyncClient(timeout=30)
        self.merchants = {} # url -> merchant info dict

    async def discover(self, merchant_url: str) -> dict:
        """Fetch a merchant's UCP profile from /.well-known/ucp."""
        resp = await self.client.get(f"{merchant_url}/.well-known/ucp")
        resp.raise_for_status()
        profile = resp.json()
        ucp = profile["ucp"]
        info = {
            "name": merchant_url.split("//")[-1],
            "mcp_endpoint": ucp["services"]["dev.ucp.shopping"][0]["endpoint"],
            "capabilities": list(ucp.get("capabilities", {}).keys()),
            "payment_handlers": list(ucp.get("payment_handlers", {}).keys()),
        }
        self.merchants[merchant_url] = info
        return info

    async def mcp_call(
        self, merchant_url: str, tool_name: str, arguments: dict
    ) -> dict:
        """Call a merchant's MCP tool via JSON-RPC 2.0."""
        merchant = self.merchants[merchant_url]
        resp = await self.client.post(
            merchant["mcp_endpoint"],
            json={
                "jsonrpc": "2.0",
                "id": uuid.uuid4().hex,
                "method": "tools/call",
                "params": {"name": tool_name, "arguments": arguments},
            },
        )
        resp.raise_for_status()
        data = resp.json()
        if "error" in data:
            raise Exception(f"MCP error: {data['error']}")
        return data.get("result", {})

    async def close(self):
        await self.client.aclose()

```

建立 **ap2.py** :

```

"""AP2 mandate handler — creates and signs payment mandates."""

import uuid
import hashlib

class AP2Handler:
    def process_cart_mandate(self, checkout_response: dict) -> dict | None:
        """Extract the merchant-signed CartMandate from a checkout response.

        The CartMandate is the merchant's cryptographic price guarantee —
        it locks the total so it can't change between checkout and payment.
        """
        return checkout_response.get("ap2", {}).get("cart_mandate")

    def create_payment_mandate(
        self, cart_mandate: dict, payment_method: str = "card"
    ) -> dict:
        """Create and sign a PaymentMandate authorizing payment.

        References the merchant's CartMandate and adds user authorization.
        Together they form a two-party agreement: merchant guarantees price,
        user authorizes charge.
        """
        contents = cart_mandate["contents"]
        mandate_id = uuid.uuid4().hex

        return {
            "mandate_id": mandate_id,
            "cart_reference": contents["id"],
            "merchant": contents["merchant_name"],
            "total": contents["total"],
            "payment_method": payment_method,
            "user_authorization": self._sign(mandate_id, contents["id"]),
        }

    def _sign(self, mandate_id: str, checkout_id: str) -> str:
        """Sign the mandate. Production uses real crypto (sd-jwt-vc)."""
        payload = f"{mandate_id}:{checkout_id}"
        return hashlib.sha256(payload.encode()).hexdigest()

```

模擬商家

建立 `merchants.py` :

```
"""Mock UCP merchant servers — two theaters with different capabilities."""
```

```
import uuid
import time
import multiprocessing
from datetime import datetime, timezone, timedelta
```

```
import uvicorn
from fastapi import FastAPI
```

```
# — Theater data —
```

```
THEATERS = {
    8081: {
        "name": "Meridian Cinemas",
        "movies": [
            {
                "id": "opp",
                "title": "Oppenheimer",
                "categories": ["Drama", "History"],
                "showtimes": [
                    {"id": "st_opp_7pm_imax", "format": "IMAX", "time": "7:00 PM", "price":
2200, "seats": 45},
                    {"id": "st_opp_930pm", "format": "Standard", "time": "9:30 PM", "price":
1500, "seats": 80},
                ],
            },
            {
                "id": "dune3",
                "title": "Dune: Part Three",
                "categories": ["Sci-Fi", "Action"],
                "showtimes": [
                    {"id": "st_dune_8pm_imax", "format": "IMAX", "time": "8:00 PM", "price":
2200, "seats": 30},
                ],
            },
        ],
        "discounts": {},
    },
    8082: {
        "name": "StarLight Theaters",
        "movies": [
            {
                "id": "opp",
                "title": "Oppenheimer",
                "categories": ["Drama", "History"],
                "showtimes": [
                    {"id": "st_opp_6pm_atmos", "format": "Dolby Atmos", "time": "6:00 PM",
"price": 1800, "seats": 60},
                ],
            },
            {

```

```

        "id": "spider",
        "title": "Spider-Verse",
        "categories": ["Animation", "Action"],
        "showtimes": [
            {"id": "st_spider_4pm", "format": "Standard", "time": "4:00 PM", "price":
1200, "seats": 100},
            ],
        },
    ],
    "discounts": {},
},
}

```

```

def create_app(port):
    theater = THEATERS[port]
    app = FastAPI()
    sessions = {}

    # ——— UCP Discovery endpoint ———
    @app.get("/.well-known/ucp")
    def discovery():
        caps = {
            "dev.ucp.shopping.catalog.search": [{"version": "2026-01-15"}],
            "dev.ucp.shopping.catalog.lookup": [{"version": "2026-01-15"}],
            "dev.ucp.shopping.checkout": [{"version": "2026-01-15"}],
            "dev.ucp.shopping.ap2_mandate": [{"version": "2026-01-15"}],
        }

        return {
            "ucp": {
                "version": "2026-01-15",
                "services": {
                    "dev.ucp.shopping": [
                        {"version": "2026-01-15", "transport": "mcp",
                         "endpoint": f"http://localhost:{port}/mcp"}
                    ]
                },
                "capabilities": caps,
                "payment_handlers": {
                    "com.example.card": [
                        {"id": f"card_{port}", "version": "2026-01-15",
                         "available_instruments": [{"type": "card"}], "config": {}}
                    ]
                },
            },
        }

    # ——— MCP JSON-RPC endpoint ———
    @app.post("/mcp")
    def mcp(body: dict):
        tool = body["params"]["name"]

```

```

args = body["params"].get("arguments", {})
rid = body.get("id", "1")

if tool == "search_catalog":
    q = args.get("query", "").lower()
    hits = [m for m in theater["movies"]
              if not q or q in m["title"].lower()
              or any(q in c.lower() for c in m["categories"])]
    return _ok(rid, {"products": [_product(m) for m in hits]})

if tool == "lookup_catalog":
    mid = args.get("product_id") or (args.get("ids", [None])[0])
    movie = next((m for m in theater["movies"] if m["id"] == mid), None)
    if not movie:
        return _err(rid, "Not found")
    return _ok(rid, {"products": [_product(movie)]})

if tool == "create_checkout":
    co = args.get("checkout", {})
    sid = f"chk_{uuid.uuid4().hex[:12]}"
    items, subtotal = [], 0
    for li in co.get("line_items", []):
        st = _find_showtime(li["item"]["id"])
        if not st:
            continue
        mv = _find_movie(li["item"]["id"])
        qty = li.get("quantity", 1)
        amt = st["price"] * qty
        subtotal += amt
        items.append({
            "id": f"li_{uuid.uuid4().hex[:8]}",
            "item": {
                "id": st["id"],
                "title": f"{mv['title']} - {st['format']} {st['time']}",
                "price": st["price"],
            },
            "quantity": qty,
            "totals": [{"type": "subtotal", "amount": amt}],
        })
    tax = int(subtotal * 0.08)
    total = subtotal + tax
    session = {
        "id": sid,
        "status": "ready_for_complete",
        "currency": "USD",
        "line_items": items,
        "totals": [
            {"type": "subtotal", "display_text": "Subtotal", "amount": subtotal},
            {"type": "tax", "display_text": "Tax", "amount": tax},
            {"type": "total", "display_text": "Total", "amount": total},
        ],
        "metadata": {"theater_name": theater["name"]},
    }

```

```

        "ap2": {
            "cart_mandate": {
                "contents": {
                    "id": sid,
                    "merchant_name": theater["name"],
                    "total": {
                        "label": "Total",
                        "amount": {"currency": "USD", "value": total / 100},
                    },
                    "cart_expiry": (
                        datetime.now(timezone.utc) + timedelta(minutes=10)
                    ).isoformat(),
                },
                "merchant_authorization": f"mock_merchant_sig_{sid}",
            },
        },
    }
    sessions[sid] = session
    return _ok(rid, session)

if tool == "get_checkout":
    sid = args.get("checkout", {}).get("id") or args.get("id")
    return _ok(rid, sessions.get(sid, {"error": "not_found"}))

if tool == "complete_checkout":
    co = args.get("checkout", {})
    sid = co.get("id")
    session = sessions.get(sid)
    if not session:
        return _err(rid, "Not found")
    session["status"] = "completed"
    session["order"] = {
        "id": f"ord_{uuid.uuid4().hex[:8]}",
        "created_at": datetime.now(timezone.utc).isoformat(),
        "tickets": [
            {
                "movie": li["item"]["title"],
                "quantity": li["quantity"],
                "ticket_code": uuid.uuid4().hex[:8].upper(),
            }
            for li in session["line_items"]
        ],
    }
    session["ap2"]["payment_mandate_verified"] = True
    return _ok(rid, session)

return _err(rid, f"Unknown tool: {tool}")

def _ok(rid, result):
    return {"jsonrpc": "2.0", "id": rid, "result": result}

```



```

def _err(rid, msg):
    return {"jsonrpc": "2.0", "id": rid, "error": {"code": -32000, "message": msg}}

def _product(movie):
    return {
        "id": movie["id"],
        "title": movie["title"],
        "categories": movie["categories"],
        "variants": [
            {
                "id": st["id"],
                "selected_options": [
                    {"name": "format", "value": st["format"]},
                    {"name": "time", "value": st["time"]},
                ],
                "price": {"amount": st["price"], "currency": "USD"},
                "availability": {"available": True, "seats_available": st["seats"]},
            }
            for st in movie["showtimes"]
        ],
    }

def _find_showtime(sid):
    return next(
        (st for m in theater["movies"] for st in m["showtimes"] if st["id"] == sid),
        None,
    )

def _find_movie(sid):
    return next(
        (m for m in theater["movies"] for st in m["showtimes"] if st["id"] == sid),
        None,
    )

return app

def _run(port):
    uvicorn.run(create_app(port), host="0.0.0.0", port=port, log_level="warning")

if __name__ == "__main__":
    for port in THEATERS:
        multiprocessing.Process(target=_run, args=(port,), daemon=True).start()
    print("Merchants running: Meridian (:8081), StarLight (:8082)")
    print("Press Ctrl+C to stop")
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        pass

```

這會建立兩個 FastAPI 伺服器，每個伺服器都有兩個端點：

- **GET** `/.well-known/ucp` - UCP 探索。傳回商家的功能、MCP 端點網址和接受的付款方式。
- **POST** `/mcp` - MCP (Model Context Protocol) 作業。處理目錄搜尋、結帳、折扣和付款的 JSON-RPC 2.0 呼叫。

在新終端機中啟動商家，商家必須保持運作狀態：

```
cd agent_payments
source .venv/bin/activate
python merchants.py
```

如下所示：

```
Merchants running: Meridian (:8081), StarLight (:8082)
```

返回第一個終端機，驗證 UCP 探索：

```
curl -s http://localhost:8081/.well-known/ucp | python -m json.tool
```

您應該會看到商家的功能、MCP 端點網址和付款處理常式。

步驟 8 · 約 2 分鐘

8. 使用 ADK Web 執行代理

我們來使用 ADK CLI 內建的網頁式 UI！這會在瀏覽器中提供即時通訊介面，並自動處理工具確認提示。

您的專案現在看起來應該會像這樣：

```
agent_payments/  
├── merchants.py      # Mock UCP merchants  
├── ucp.py            # UCP client helper  
├── ap2.py            # AP2 mandate handler  
├── tools.py          # Agent tools  
├── agent.py          # Agent definition  
└── pyproject.toml
```

立即體驗

在目前的終端機中，前往上層目錄 (`agent_payments` 上方的一個資料夾)，然後啟動 ADK 網頁 UI：

```
cd ../  
adk web --allow_origins '*'
```

輸出內容應會顯示伺服器正在執行，並提供網址 (通常是 `http://localhost:8000` 或類似網址)。

與服務專員交談

1. 在瀏覽器中開啟 `adk web` 提供的網址。
2. 畫面上會顯示聊天介面。
3. 嘗試詢問：「目前上映中的電影？」
4. 服務專員會在幕後探索電影院和搜尋目錄，並透過 UCP 彙整兩家商家的結果。
5. 要求訂票：「預訂晚上 7 點的《奧本海默》電影票 2 張」。
6. 當代理程式嘗試呼叫 `complete_purchase` 時，請注意 ADK 網頁介面會彈出**確認對話方塊**或資訊卡！
7. 如要授權交易，請在對話中回覆這段確切的 JSON 字串： `{"confirmed": true}`。
8. 服務專員會完成購買程序，並將訂單確認資訊連同票券代碼傳送給您！

幕後到底發生了什麼事？

1. `create_checkout` → 商家傳回 AP2 **CartMandate** (已簽署的價格鎖定)
 2. `complete_purchase` → 建立 **PaymentMandate**、簽署 (模擬 SHA-256)，並將兩個授權書傳送給商家
 3. 商家驗證兩個簽章 → 核發票券 (模擬)
-

9. 清除所用資源

為避免本機伺服器持續運作，請清理資源：

1. 在執行 `adk web` 的終端機中，按下 **Ctrl+C** 鍵停止代理程式伺服器。
2. 在執行 `python merchants.py` 的終端機中，按下 **Ctrl+C** 鍵即可停止模擬商家。
3. 在兩個終端機中執行下列指令，停用虛擬環境：

```
deactivate
```

4. (選用) 如果您為這個程式碼研究室建立了新的 Google Cloud 雲端專案，並想刪除該專案，請執行下列指令：

```
gcloud projects delete $GOOGLE_CLOUD_PROJECT
```

10. 恭喜！🎉

您已建構 ADK 代理，可使用 UCP 和 AP2 探索商家、瀏覽目錄及完成購買。

您學到的內容

在本程式碼研究室中，您建構了可處理安全商務流程的 ADK 代理程式。以下簡要說明您建構的內容，以及套用的重要概念：

建構內容：

- **5 個代理工具**，可包裝 UCP 和 AP2 作業，包括探索、目錄搜尋、結帳和付款。
- **AP2 委託書簽署**：CartMandate (商家價格鎖定) + PaymentMandate (使用者授權)。
- **多商家搜尋**：一個代理程式查詢多個劇院，並合併結果。

重要概念：

通訊協定	用途	運作方式
UCP 探索	代理程式會尋找商家及其服務	<code>/.well-known/ucp</code> → 功能、MCP 端點、付款方式
UCP MCP	代理程式瀏覽目錄、建立結帳程序	對商家 MCP 端點的 JSON-RPC 2.0 呼叫
AP2 CartMandate	商家鎖定報價	由商家簽署，包含總金額和有效期限
AP2 PaymentMandate	使用者授權扣款	使用者簽署，參照 CartMandate

正式版有何不同？

本程式碼研究室會使用模擬。實際運作中：

- **UCP 探索**會根據登錄檔解析，而不是硬式編碼的 localhost 網址
- **MCP 端點**由實際商家代管，採用相同的 JSON-RPC 2.0 通訊協定，提供真實的商品目錄
- **AP2 授權**使用 sd-jwt-vc 簽署，而非 SHA-256 雜湊
- **付款授權**：使用 AP2 Wallet SDK，並顯示使用者同意提示
- **前端**會將工具結果算繪為豐富的使用者介面 (產品格線、結帳摘要、確認卡)

後續步驟

- 探索 [AP2 通訊協定規格](#)，並建構自己的付款功能代理
- 為商家導入 [UCP](#)，啟用代理商務
- 透過 [A2A](#) 連線 AP2，進行多代理商務工作流程
- 瞭解加密貨幣付款的 [AP2 x402 擴充功能](#)

參考文件

- [AP2 通訊協定說明文件](#)

- [UCP 開發人員指南](#)
 - [AP2 GitHub 存放區](#)
 - [ADK 說明文件](#)
 - [UCP 規格](#)
-